

MSPR-Deploy

Orchestration Docker Compose de la plateforme **MSPR HealthAI Coach**.

Deux modes, un seul script :

- **Mode prod** : `./bootstrap.sh` pull les images publiees sur GHCR. Zero clone supplementaire, ideal pour evaluation et demo.
- **Mode dev** : `./bootstrap.sh --dev` clone les 8 depots sources sur leurs branches actives et build localement.

Prerequis

- Docker Engine 24+ (Linux/macOS/Windows)
- Docker Compose v2 (inclus dans Docker Desktop)
- ~10 Go d'espace disque (pour Ollama gemma3:4b notamment)
- Ports libres : 3000 , 5173 , 5433 , 5434 , 8001 , 8002 , 8080 , 27018
- Pour le mode dev : git , bash , openssl

Demarrage en une commande

```
git clone https://github.com/ArthurPoncin/MSPR-Deploy.git
cd MSPR-Deploy
./bootstrap.sh          # mode prod (default)
# ou
./bootstrap.sh --dev    # mode dev (clone sources + build local)
```

Le script :

1. (mode dev uniquement) clone les 8 depots `whitefoxyt/MSPR-HealthAI-Coach-*` dans `sources/` sur leurs branches actives (`main` , `master` , `dev`) ;
2. cree `.env` depuis `.env.example` si absent et genere `BETTER_AUTH_SECRET` ;
3. dechiffre `secrets/resend.enc` (demande la passphrase fournie par l'equipe) ;
4. lance `docker compose up -d` (mode prod) ou `docker compose -f docker-compose.dev.yml up -d --build` (mode dev).

Pour eviter la saisie interactive, passer la passphrase en variable d'environnement :

```
MSPR_PASS="<passphrase fournie par l'equipe>" ./bootstrap.sh
```

Au premier lancement, Docker pull (ou build) les 8 services puis Ollama telecharge `gemma3:4b` (~3 Go). Comptez 5 a 10 minutes selon la connexion.

Pour ne pas demarrer la stack a la fin : `./bootstrap.sh --no-up` (ou `./bootstrap.sh --dev --no-up`).

Acces aux services

Service	URL	Description
Frontend Vue 3	http://localhost:5173	Dashboard admin / data scientist
API Spring Boot	http://localhost:8080/api/swagger-ui.html	Swagger UI
Service Auth (Bun)	http://localhost:3000	Endpoints better-auth
IA Nutrition (FastAPI)	http://localhost:8001/docs	Classification aliments + plan repas
Reco Fitness (FastAPI)	http://localhost:8002/docs	Recommandations fitness
PostgreSQL metier	localhost:5434	base <code>healthai</code>
PostgreSQL auth	localhost:5433	base <code>auth_db</code>
MongoDB reco	localhost:27018	base <code>reco_fitness</code>

Verifier que tout est sain

```
docker compose ps
curl http://localhost:8080/api/actuator/health
curl http://localhost:8001/health
curl http://localhost:8002/health
```

Commandes utiles

Les commandes ci-dessous sont en mode prod (compose file par default). En mode dev, ajouter `-f docker-compose.dev.yml` apres `docker compose`.

```

# Suivre les logs d'un service
docker compose logs -f api

# Arrêter sans supprimer les données
docker compose down

# Reset complet (efface volumes : BDD, Mongo, Ollama)
docker compose down -v

# Mode prod : forcer le pull des dernières images
docker compose pull && docker compose up -d

# Mode dev : rebuild après modification d'un service source
docker compose -f docker-compose.dev.yml up -d --build api

```

Configuration

Le `.env.example` documente chaque variable. Les seuls champs obligatoires sont :

- `BETTER_AUTH_SECRET` : secret HMAC partagé entre `auth` et `api`, généré automatiquement par `bootstrap.sh`
- `RESEND_API_KEY` : clé Resend pour les emails. Une clé de demo est chiffrée dans `secrets/resend.enc` ; `bootstrap.sh` la déchiffre via la passphrase fournie par l'équipe lors de la soutenance. Pour un déploiement autonome, renseigner directement la valeur dans `.env` (le déchiffrement est alors sauté).

Les autres variables ont des valeurs par défaut dans le compose et restent commentées.

Architecture

8 microservices indépendants connectés via le réseau Docker `mspr_data_network`.

```

front ─┐
      │→ api (Spring Boot, JWT verify) → db (PostgreSQL métier)
auth ─┐  ai-nutrition (FastAPI) ────→ ollama (gemma3:4b, réseau isolé)
      │  reco-fitness (FastAPI) ────→ mongo (PostgreSQL + MongoDB)
      └→ auth-db (PostgreSQL auth)
          ^
          └─ auth-migrate (Drizzle, run once)

etl (Python, cron 0 2 * * *) ────→ db

```

Depots sources

Les images GHCR sont buildées depuis ces depots publics. Le mode dev les clone dans `sources/`.

Service	Branche active	Repository
<code>mshr-db</code>	<code>main</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-BDD
<code>mshr-auth</code>	<code>main</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-Auth
<code>mshr-api</code>	<code>main</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-API
<code>mshr-etl</code>	<code>master</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-ETL
<code>mshr-front</code>	<code>dev</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-Dahsboard
<code>mshr-ai-nutrition</code>	<code>master</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-AI-Nutrition
<code>mshr-reco-fitness</code>	<code>master</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-Reco-Fitness-
<code>mshr-mongodb</code>	<code>master</code>	https://github.com/whitefoxyt/MSPR-HealthAI-Coach-MongoDB

Monitoring / observabilite

Une stack d'observabilite (Prometheus, Grafana, Loki, Alertmanager) est fournie dans l'overlay `docker-compose.monitoring.yml`. Elle s'ajoute a la stack applicative sans la modifier.

```
# Mode prod
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d

# Mode dev
docker compose -f docker-compose.dev.yml -f docker-compose.monitoring.yml up -d
```

Outil	URL	Role
Grafana	http://localhost:3001	Tableaux de bord (admin / admin par default)
Prometheus	http://localhost:9090	Metriques + alertes
Alertmanager	http://localhost:9093	Alertes declenchees
Loki	http://localhost:3100	Logs centralises (via Grafana)

Donnees collectees : metriques applicatives (API Spring via Actuator, FastAPI et Auth via `/metrics`), metriques conteneurs (cAdvisor), hote (node-exporter), PostgreSQL metier + auth et MongoDB (exporters), et logs de tous les conteneurs (Promtail vers Loki). Liste exhaustive : voir `monitoring/README.md`.

Configurations multi-environnement

Trois configurations (complete, offline, performance) sont documentees dans `CONFIGS.md`.

```
# Complete (tous services + monitoring)
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d

# Offline (sans internet : Ollama local, Auth sans email)
docker compose -f docker-compose.yml -f docker-compose.offline.yml up -d

# Performance (limites CPU/RAM ; voir CONFIGS.md pour le demarrage allège)
docker compose -f docker-compose.yml -f docker-compose.performance.yml up -d
```

Sauvegarde et restauration

Scripts dans `scripts/`, operant sur les conteneurs en cours d'execution :

```
./scripts/backup.sh          # dump horodate dans backups/<timestamp>/
./scripts/restore.sh         # restaure la derniere sauvegarde
./scripts/restore.sh backups/XXXX # restaure une sauvegarde precise
./scripts/clean.sh           # remise a zero (supprime les volumes) ; --dev pour l
```

`backup.sh` sauvegarde PostgreSQL metier (`healthai`), PostgreSQL auth (`auth_db`) et MongoDB (`reco_fitness`). `restore.sh` recharge ces dumps (`pg_dump --clean`, `mongorestore --drop`, donc remplace les donnees existantes). Le dossier `backups/` n'est pas versionne.

Depannage

Le pull Ollama est tres long. Le premier `docker compose up` telecharge `gemma3:4b` (~3 Go). Suivre avec `docker compose logs -f ollama`.

Erreur `BETTER_AUTH_SECRET is required`. Generer puis renseigner la valeur dans `.env` : `openssl rand -base64 64`.

Passphrase Resend invalide. Verifier la passphrase fournie par l'equipe ou renseigner manuellement `RESEND_API_KEY` dans `.env` (cle Resend personnelle, obtenue via <https://resend.com/api-keys>).

Conflit de port. Adapter la partie hote des mappings dans `docker-compose.yml` (ex : `"15173:5173"`).

Reset complet. `docker compose down -v` supprime tous les volumes ; le prochain `up` repart d'une base vierge et l'ETL recharge les datasets.

Re-cloner les sources (mode dev). Supprimer un sous-dossier de `sources/` et relancer `./bootstrap-dev.sh`.